

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



FINAL REPORT

JumpToTheEndProject

Course: OBJECT-ORIENTED-PROGRAMMING- IT079IU

ADVISOR:

Assoc. Prof. Nguyễn Thanh Tùng

ABSTRACT

Jump Back is a 2D platformer game that blends contemporary gameplay principles with aspects of vintage side-scrolling games. Players take control of a brave character in this colorful and lively game as they try to stop an evil sorcerer from destroying their community.

There are several stages in the game, and each has its own special difficulties and barriers. To advance through these stages, players must acquire power-ups, vanquish opponents, and solve riddles. Because to the responsive and easy-to-use controls, players of all skill levels may enjoy this game.

The dynamic difficulty system of Artifact game is one of its main characteristics. The game offers a difficult yet equitable experience by adjusting to the player's skill level. The game's richness and replayability are further increased by the assortment of mini-games and side missions it offers.

To sum up Jump Back is an entertaining and captivating 2D platformer game that provides a distinctive fusion of traditional and contemporary gameplay components. Adventure Quest is likely to keep gamers of all ages entertained for hours on end with its simple controls, varied difficulty structure, and endearing artwork

Group 6 Project's Member

No.	Full Name	ID	Role	Contribution
1	Phan Minh Nguyên	ITITUN21143	Member	25%
2	Bùi Duy Đức	ITITIU22033	Member	25%
3	Nguyễn Hoàng Giang	ITITIU22046	Member	25%
4	Bùi Gia Bảo	ITITIU22019	Leader	25%

Table 1: The contribution and information of all project members

TABLE OF CONTENTS

Group 6 Project's Member.....	4
Chapter I. Introduction:	5
1. Plan	5
2. Table	5
Chapter II. Requirements 4 Properties of OOP	6
1. Encapsulation	6
2. Abstraction:	6
Chapter III. Design and Diagram	7
1. Audio and Effect:	7
2. Entities:.....	8
3. Gamestates	14
4. Inputs:.....	23

5. Levels:	24
6. Main:.....	15
7. Object:	15
8. UI:	19
Chapter 4. Conclusion:	20
Chapter 5. GITHUB:.....	20

LIST OF FIGURES

Figure 1 Entities	7
Figure 2: Gamestates	12
Figure 3: Inputs	13
Figure 4: GameObject	14
Figure 5: GameObject	14

I. Introduction

1. Plan

An overview of an artifact project is what this paper aims to convey. Stakeholders engaged in its creation, implementation, and maintenance are the target audience for this paper, which includes design, implementation, and assessment. For anyone who wants to know more about Artifact Project, the report is a useful reference.

2. Table of timeline

Week	Task
1 - 4	Learn Basics of Javascript-ES6 and decide a type of game, Gameplay Mechanics, Art Style.
5 - 8	Design characters, level and use interface
9 - 12	- Implement the gameplay mechanics, character movements, collision detection, etc. - o - Create or source the art assets for your game, including characters, backgrounds, and UI elements.
13 (Deadline)	Play the game yourself and make adjustments based on our experience. Have others play your game and provide feedback

II. Requirements and Properties of OOP

1. Requirements

The process of making a 2D game is thrilling and starts with a well-defined concept and design that specify the genre, plot, characters, and aesthetic.

A thorough grasp of a programming language appropriate for game creation, such as Python and Pygame, JavaScript and Phaser, or C# and Unity, IntelliJ and GIMP, is necessary for this procedure. By offering necessary tools and frameworks, game engines such as Godot, Unity, or GameMaker may greatly expedite the creation process.

Characters, locations, and in-game items are all rendered in two dimensions utilizing graphic design tools, which adds to the game's visual attractiveness. Sound effects and music are added to further improve the immersive game experience.

Thorough testing is essential to guarantee seamless gaming and to find and fix any faults. When a game is flawless, it's prepared for release on an appropriate platform. This may be a web-based platform for browser games, a PC platform like Steam, or a mobile app store. Making a 2D game only takes time, patience, and a will to learn, but it can be a really fulfilling process despite the difficulties.

2. Properties of OOP

A. Encapsulation:

- Encapsulation in our game project using classes like Player, Enemy, and Game.
- Attributes and methods are hidden.

- For example, the Player class has specific attributes.
- Getters and setters.

B.Abstraction:

- Create subclasses for code reuse and hierarchy.
- Enemy class.
- The GameState interface.
- Abstraction enhanced modularity and maintainability of shared features.

C.Inheritance:

- Create subclasses for code reuse and hierarchy.
- For example, MapObject and GameState
- Reducing code duplication.
- Promoting modular code

D.Polymorphism:

- The EnemyManager class contains 2 checkEnemyHit methods.
- Depend on the type of collision.

III. Design and Implementation

1.Audio and Effect

The **AudioPlayer** class is well-structured and provides comprehensive functionality for managing background music and sound effects in a game. Here are some observations.

The **DialogueEffect** class is a straightforward implementation for handling animated dialogue effects in a game. It manages the position, type, animation state, and activity status of a dialogue effect. Below are some observations and suggestions for improvement:

The **Rain** class simulates a rainfall effect using particles. It uses an array of `Point2D.Float` objects to represent rain drops and updates their positions frame by frame to create a dynamic effect. Below are observations, potential optimizations, and suggestions for improvement.

1. **Core Functionality:**

- Initializes rain drops at random positions.
- Moves the drops downward at a constant speed (`rainSpeed`).
- Resets drops to a random x-position and above the screen when they fall off the screen.

2. **Graphics Rendering:**

- Uses a preloaded rain particle image for rendering.
- Dynamically offsets the x-coordinate of drops based on the level offset (`xLvIOffset`).

3. **Performance Considerations:**

- Managing 1000 particles with constant updates can strain lower-end systems, especially if many other game elements are rendered.

2. Entities:

A. Crabby:

The **Crabby** class is a specialized enemy implementation with distinct behaviors for movement, attacking, and reacting to environmental changes. Here's an analysis and suggestions for refinement and improvement.

1. **Interaction with Gameplay:**

- Uses the `Playing` class to interact with the player and spawn dialogues (e.g., exclamation points).

2. Animation and Effects:

- Animations are updated per frame, and specific frames trigger actions like checking for player hits.

3. Environmental Awareness:

- Incorporates collision and "floor check" logic, ensuring the enemy stays grounded or responds to being airborne.

B. Enemy:

The **Enemy** class is a comprehensive abstract base for enemy behaviors in your game. It effectively encapsulates the core mechanics for movement, attacks, animation, and interactions with the environment. Below is a breakdown of its strengths and areas for improvement, followed by suggestions for enhancements.

1. State Management:

- Manages enemy states like IDLE, RUNNING, ATTACK, HIT, and DEAD through transitions.
- Incorporates logic to reset animation states and handle specific scenarios for different enemy types (CRABBY, SHARK, PINKSTAR).

2. Environmental Awareness:

- Checks for floor, water, air conditions, and spikes to adjust enemy behavior accordingly.
- Uses helper methods (IsFloor, IsEntityInWater, CanMoveHere) to ensure enemies react dynamically to their surroundings.

3. Player Interaction:

- Implements methods like canSeePlayer, isPlayerInRange, and isPlayerCloseForAttack to control how enemies engage with the player.

4. Animation Control:

- Handles animation updates with fine-grained control, supporting state-specific behaviors (e.g., looping attack animations for PINKSTAR).

5. Modular and Reusable:

- Provides a robust framework for extending enemy types, allowing for custom behaviors while sharing common functionality.

C. EnemyManager:

The **EnemyManager** class effectively handles the management of different types of enemies (Crabby, Pinkstar, and Shark). It covers updating, drawing, checking interactions, and resetting enemy states. Below is an evaluation of its current implementation, highlighting its strengths, weaknesses, and opportunities for refactoring.

1. Centralized Management:

- Manages all enemy types from a single location, which simplifies gameplay control and synchronization.

2. Separation of Logic:

- The draw, update, and checkEnemyHit methods are neatly separated, keeping responsibilities clear.

3. Dynamic Image Loading:

- Uses sprite atlases with configurable sizes for efficient image handling.

4. Scalable Structure:

- Supports multiple enemy types by iterating over separate lists for each type, which is extensible for additional enemy types.

5. Reset Functionality:

- Provides a resetAllEnemies method to cleanly reset the game state for all enemies.

D. Entity:

1. Attributes:

- **Position and Dimensions:** x, y, width, height.

- **Hitbox & Attack Box:** Represented as Rectangle2D.Float objects for collision and attack detection.
 - **Animation:** Controlled by aniTick and aniIndex.
 - **State Management:** Tracks the entity's current state using state.
 - **Movement & Physics:** Attributes like airSpeed, inAir, and walkSpeed handle movement dynamics.
 - **Health:** Includes maxHealth and currentHealth.
2. **Pushback Logic:**
 - The method pushBack handles entity pushbacks based on the direction and checks if the move is valid using CanMoveHere.
 3. **Rendering:**
 - drawHitbox and drawAttackBox methods provide visual debugging tools for hitbox and attack box.
 4. **Initialization:**
 - initHitbox sets up the entity's hitbox with scaling based on the game.
 5. **State & Animation Management:**
 - newState initializes a new state and resets animation parameters.

E. Pinkstar:

The **Pinkstar** class is a specific type of Enemy in your game, implementing unique behavior and interaction logic. Here's a detailed analysis and suggestions for improvement:

F. Player:

This code represents a **Player** entity in a game, implementing core mechanics such as movement, jumping, attacking, knockback, and status updates. Below is a breakdown of key features and some observations:

1. Movement & Physics:

- **Walking:** Controlled by left and right flags. Adjusts xSpeed and determines facing direction using flipX and flipW.
- **Jumping:**
 - Features charged jumps (jumpHeld) to calculate jump strength based on jumpChargeTime.
 - Introduces a maxJumpChargeTime to cap jump power.
- **Knockback:** Applies force and duration-based movement in response to an external effect, using knockbackStartTime and knockbackDirection.

2. Combat:

- **Attacking:**
 - Regular attacks controlled by attacking flag.
 - Power attacks consume powerValue and amplify attack effects.
 - Attack areas are dynamically adjusted based on the facing direction.
- **Health & Power Management:**
 - Health represented by currentHealth and visualized via a health bar.
 - powerBar tracks special ability resources, regenerating over time.

3. Animation:

- **State-Driven Animation:**
 - States like IDLE, RUNNING, JUMP, FALLING, and HIT determine the active animation.
 - Animations are reset or advanced using updateAnimationTick().

4. Environmental Interactions:

- **Collision Handling:** Uses utility functions (CanMoveHere, IsEntityOnFloor) to restrict movements and enforce physics.
- **Hazards:** Checks for interaction with spikes, water, and potions.

G. Shark:

The **Shark** class represents an enemy in a game, likely a platformer or action game.

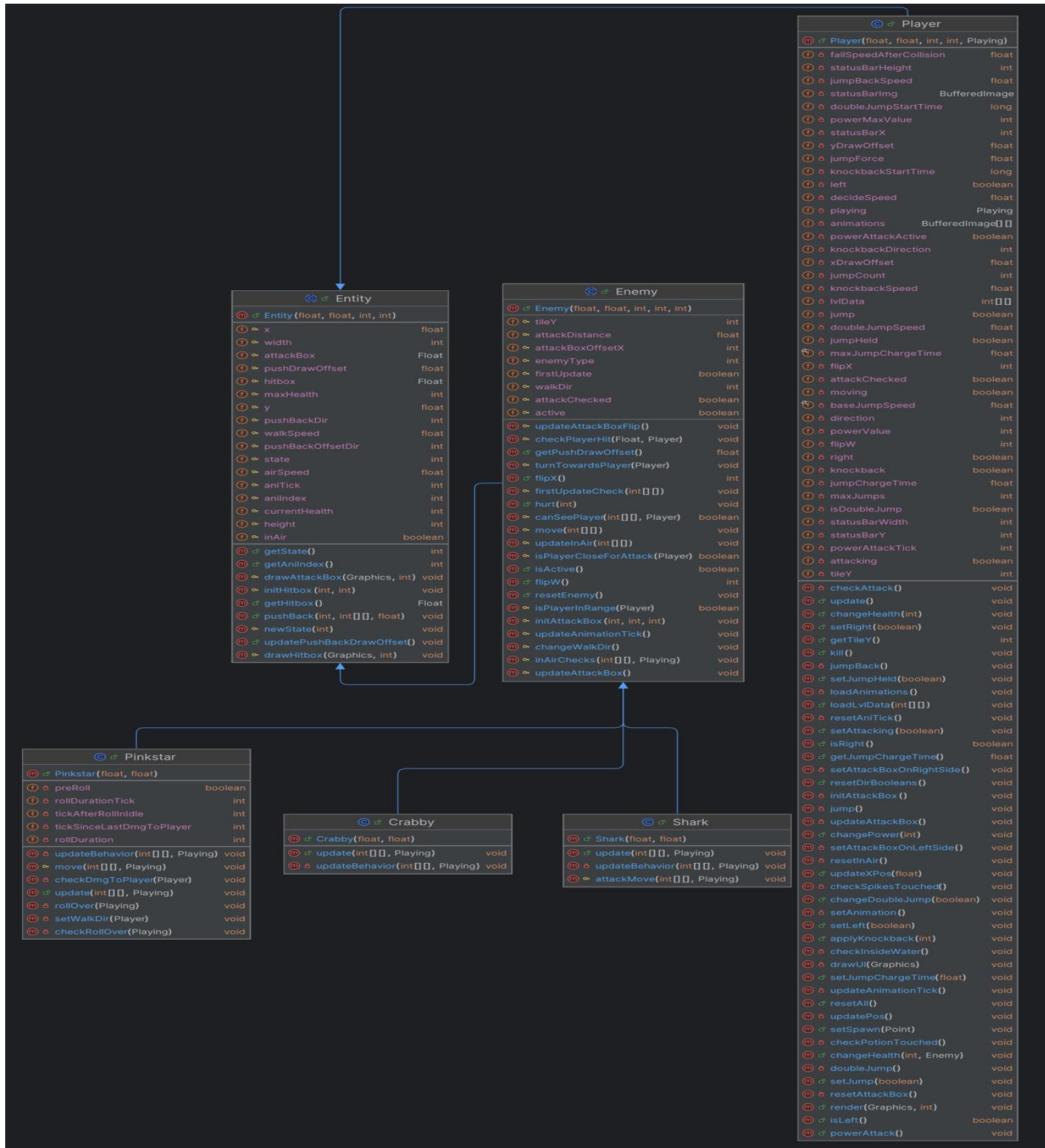


Figure 1: Entites

3. Gamestates:

A. Credits:

The **Credits** class represents the credits screen of a game. It handles the display of credits, animation of various game entities, and user interaction, such as keyboard input. Here's an analysis of its structure and functionality:

Key Concepts:

1. Inheritance:

- The class extends State and implements StateMethods, meaning it manages the state transition logic for the credits screen.

2. Entities:

- The entitiesList holds different game entities (like the player, crab, shark, and pink star), which are animated and shown during the credits sequence.

3. Background and Credits Images:

- Two main images are loaded: backgroundImage (background for the screen) and creditsImg (image for the credits content).
- The background is moved vertically using bgYFloat to create a scrolling effect.

Detailed Breakdown:

1. Constructor:

- **Credits(Game game):**
 - Initializes background and credits images.
 - Sets up the background's position and size according to the game's resolution and scale.
 - Calls loadEntities() to populate the list of entities that will be animated and displayed.

2. "loadEntities" Method:

- This method creates a list of ShowEntity objects, each representing a game character (like the player, crab, shark) with an idle animation.

- Each ShowEntity object is initialized with the idle animation frames and its position on the screen (determined by game width and height).
3. **“getIdleAni” Method:**
- This helper method extracts a series of images from a sprite atlas to create an idle animation for each entity. It takes in the sprite atlas, the number of sprites in the animation, and the width/height of each sprite.
4. **“update” Method:**
- The background (bgYFloat) scrolls vertically at a rate of 0.2 units per update, giving a scrolling effect for the credits.
 - Updates each entity in entitiesList (i.e., updates their animations).
5. **“draw” Method:**
- Draws the background and credits images, followed by each entity's current frame of animation (scaled by 4 for visual effect).
6. **“key and Mouse” Event Handlers:**
- **keyReleased(KeyEvent e):**
 - If the ESC key is pressed, it resets the background position and switches the game state back to the menu.
 - The mouse event methods are placeholders for future functionality, indicating that mouse interactions are not currently used on the credits screen.
7. **ShowEntity Class:**
- This is a helper class representing an animated entity in the credits. Each entity has an array of images (idleAnimation), which are cycled through for animation.
 - **update():** Updates the animation index by incrementing it each frame. If the animation reaches the end, it loops back to the start.
 - **draw(Graphics g):** Draws the current frame of the animation at the entity's specified position, scaled by a factor of 4.

B. GameOptions:

The **GameOptions** class manages the game's options screen, specifically for audio settings and navigating back to the main menu. This screen includes UI elements like buttons and audio sliders, and handles user interactions such as mouse and keyboard events. Here's a breakdown of its structure and functionality:

1. **Constructor:**

- **GameOptions(Game game):**
 - Calls the parent constructor (super(game)).
 - Loads the background images (loadImgs()), button (loadButton()), and initializes the audioOptions object for managing audio settings.

2. **“loadButton” Method:**

- Initializes a UrmButton at a specific location on the screen. This button serves as the "Back to Menu" button.

3. **“loadImgs” Method:**

- Loads the background images using LoadSave.GetSpriteAtlas. The background images are scaled according to the game's scale factor (Game.SCALE), and the positions of the background images are set based on the game's width and height.

4. **“update” Method:**

- Updates the button (menuB) and the audio options (audioOptions) each frame.

5. **“draw” Method:**

- Draws the background images and UI elements (menu button and audio options) to the screen.

6. **Mouse Event Handlers:**

- **mouseDragged(MouseEvent e):** Updates the audio options when the mouse is dragged.

- **mousePressed(MouseEvent e)**: Checks if the menu button is pressed and updates the button's state. If not, it forwards the event to the audio options.
- **mouseReleased(MouseEvent e)**: If the menu button is released, it navigates back to the main menu (Gamestate.MENU). Otherwise, it forwards the event to audio options.
- **mouseMoved(MouseEvent e)**: Updates the hover state of the menu button and forwards the event to the audio options.

7. Keyboard Event Handlers:

- **keyPressed(KeyEvent e)**: If the ESC key is pressed, it navigates back to the main menu (Gamestate.MENU).

8. Helper Methods:

- **isIn(MouseEvent e, PauseButton b)**: Checks if a mouse event occurs within the bounds of a specific button (PauseButton or menuB in this case).

C. GameState:

The **Gamestate** enum represents the different states or screens of the game. These states help manage the flow of the game, allowing it to transition between various menus, gameplay, options, and other screens.

How It Works:

- The enum defines the available game states, and the static state variable keeps track of which one is currently active.
- Throughout the game, the state can change based on user actions or events (e.g., pressing a button to go to the options screen or starting the game).
- The game logic or game loop will check the current Gamestate.state to decide which screen to display or which actions to take.

D. Menu:

The **Menu** class is part of a game state management system, and it represents the main menu of the game. This class is responsible for rendering the main menu screen, handling button interactions, and transitioning to different game states, such as starting the game, accessing the options, viewing credits, or quitting the game.

Breakdown of Methods:

- **loadBackground()**: Loads the background images and calculates the size and position for centering them on the screen.
- **loadButtons()**: Initializes the buttons for the main menu, positioning them vertically and associating them with their respective game states.
- **update()**: Updates each button in the menu, which likely includes animations, hover effects, and other state changes.
- **draw(Graphics g)**: Draws the background and the buttons on the screen.
- **mousePressed(MouseEvent e)**: Detects which button was clicked and sets it to be "pressed".
- **mouseReleased(MouseEvent e)**: Checks if a button was clicked, and if so, applies the corresponding game state transition.
- **mouseMoved(MouseEvent e)**: Detects if the mouse is hovering over a button and updates the mouseOver state accordingly.
- **resetButtons()**: Resets the mousePressed and mouseOver states for all buttons, ensuring they are ready for the next interaction.

E. Playing:

The **Playing** class is a core part of the game and represents the gameplay state. It contains various game mechanics and processes for player movement, game objects, and environment interactions. Below is a breakdown of some key elements and concepts from this class:

Main Methods:

- **update()**:

- The game state is updated each frame, handling different overlays (pause, game over, etc.).
 - Handles game updates like player, enemy, and level manager updates.
- **draw():**
 - Draws all the game elements: background, clouds, level, enemies, player, etc.
 - Also draws various overlays depending on the current state (paused, level completed, etc.).
- **mouseClicked() & keyPressed():**
 - Detect mouse clicks and key presses for actions like attacking or pausing the game.
- **setGameOver() / setLevelCompleted():**
 - These methods are used to change the game's state (e.g., setting the game over or level completion).
- **loadStartLevel():**
 - Initializes enemies and objects for the current level.
- **checkObjectHit() / checkEnemyHit():**
 - Used to check whether the player has hit an object or enemy (collision detection).

F. State:

The **State** class is a base class for managing different game states, such as menus and the playing state. Here's a breakdown of its purpose and key methods:

Purpose and Use Case:

- **State Management:** This class is responsible for managing and transitioning between different game states (like the main menu and the gameplay state).
- **Menu Interaction:** The `isIn` method checks whether the user clicked on a `MenuButton`, which is useful in menu screens where you need to interact with buttons.

How it Works in the Game:

- When transitioning from one state to another (e.g., from the main menu to the playing state), the `setGamestate()` method is called to:
 1. Change the current state of the game (using the `Gamestate` enum).
 2. Adjust the background music based on the state (e.g., play menu music when in the menu and level music during gameplay).

G. Statemethods:

The **Statemethods** interface defines the basic methods that any game state should implement in order to function properly within the game. It is designed to be a contract that ensures consistency across different game states. Here's a breakdown of its methods:

Methods:

1. `update()`:

- This method will be used to update the state of the game. For example, in the `Playing` state, it could update the player's position, check for collisions, update enemies, etc.

2. `draw(Graphics g)`:

- This method is responsible for drawing the graphical elements of the current state to the screen. It takes a `Graphics` object as a parameter, which can be used to draw images, shapes, and text.

3. `mouseClicked(MouseEvent e)`:

- This method handles mouse click events. It takes a `MouseEvent` as a parameter and allows the state to respond when the user clicks on the screen (e.g., clicking buttons in the main menu or interacting with objects in the game).

4. `mousePressed(MouseEvent e)`:

- This method is triggered when a mouse button is pressed down. It allows the game state to respond to a button press, often used for UI interactions.

5. mouseReleased(MouseEvent e):

- This method is triggered when a mouse button is released. It is often paired with mousePressed to handle full mouse interactions like dragging or button presses.

6. mouseMoved(MouseEvent e):

- This method is called when the mouse is moved. It is used to track mouse movement and can be useful for highlighting UI elements or updating object positions in response to mouse movement.

7. keyPressed(KeyEvent e):

- This method handles key press events. It takes a KeyEvent as a parameter and can be used to process input from the keyboard (e.g., movement controls, jumping, etc.).

8. keyReleased(KeyEvent e):

- This method is triggered when a key is released. It allows the game to respond when the player stops pressing a key (e.g., stopping movement or actions when a key is no longer held down).

Purpose:

- Consistency Across States: The interface provides a standard set of methods that must be implemented by any game state, ensuring that all states (menu, playing, paused, etc.) have the same core functionality.
- Event Handling: It facilitates event-driven programming by ensuring that all states handle mouse and keyboard events in a consistent way.
- Game Logic: The update() and draw() methods allow for the core game logic and rendering to happen, regardless of which state is active.

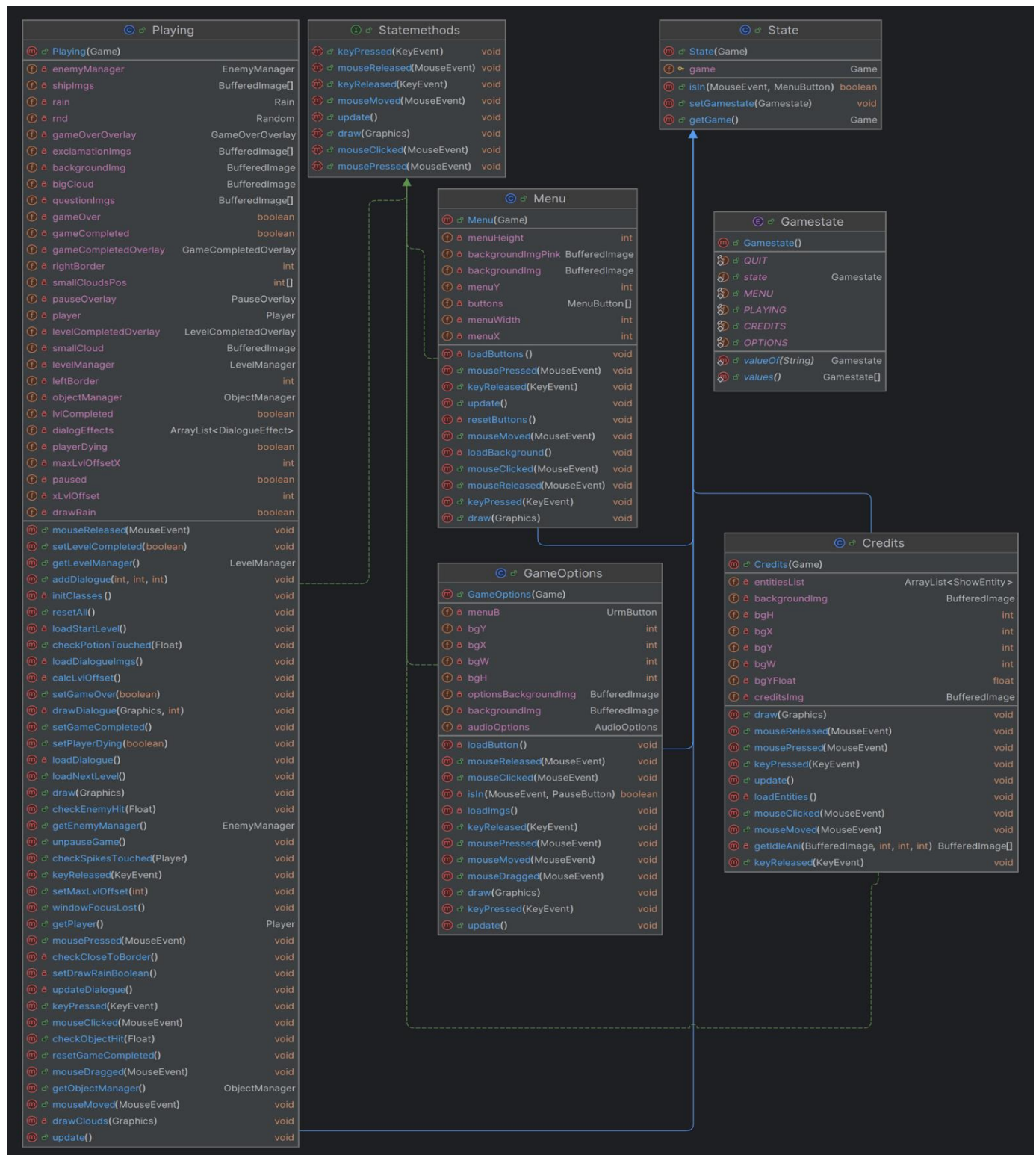


Figure 2: GameStates

4. Inputs:

The **KeyboardInputs** class handles keyboard input events for different game states. It implements the `KeyListener` interface, which requires the implementation of three methods: `keyPressed`, `keyReleased`, and `keyTyped`. The class processes key events based on the current game state and routes them to the appropriate game state class for further handling.

The **MouseInputs** class handles mouse events for different game states (e.g., Menu, Playing, Options). It implements both `MouseListener` and `MouseMotionListener` interfaces, enabling it to respond to mouse actions like clicks, movements, and dragging. The class routes each event to the appropriate game state based on the current `Gamestate.state`.

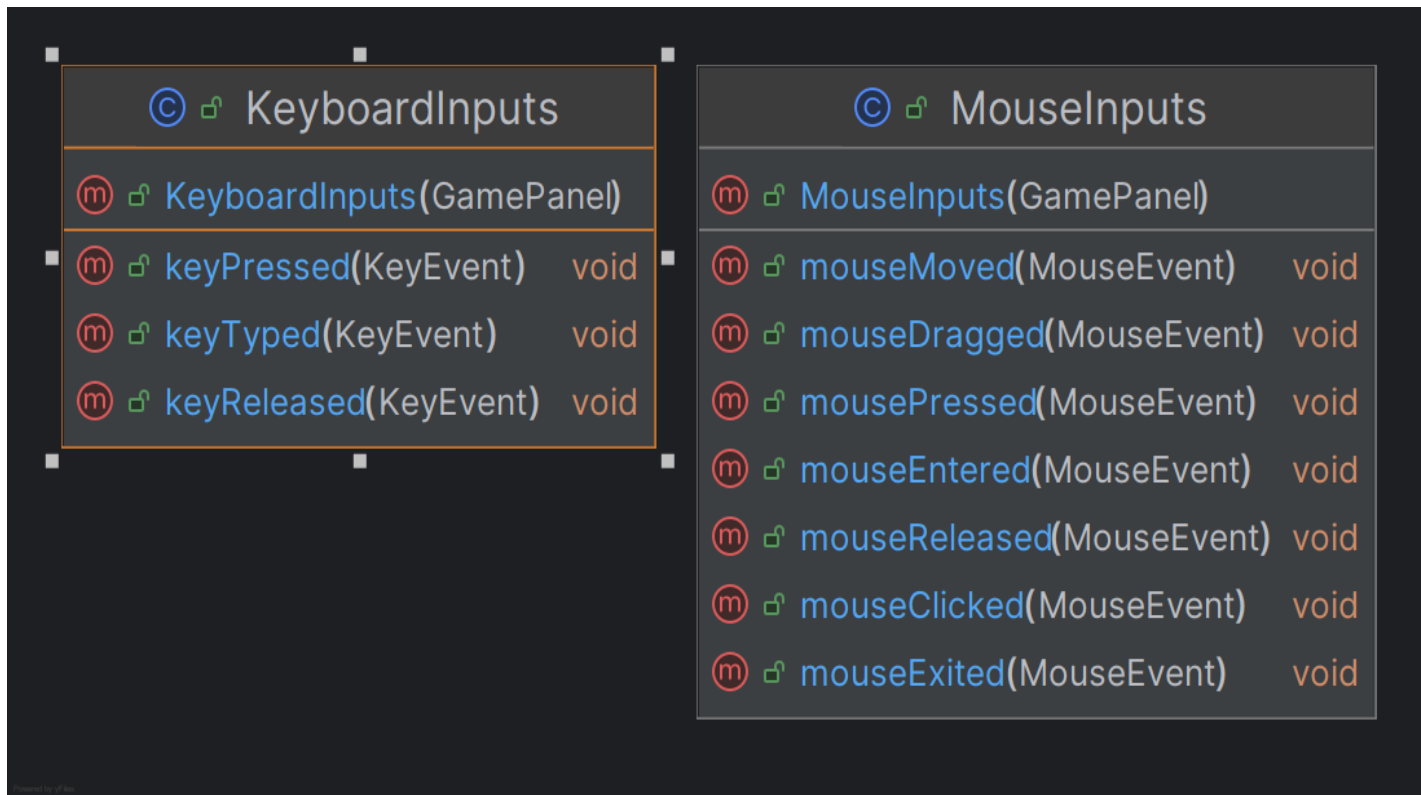


Figure 3: Inputs

5. Levels:

The **Level** class is a core component that handles the creation and management of the level's data, including entities, objects, and the background. It loads the level from a `BufferedImage`, where the pixel colors represent different elements in the game world, such as tiles, entities, and objects. Here's a breakdown of the key parts of the class:

The **LevelManager** class is a central component in managing and rendering game levels in a platformer-style game. It handles the loading and rendering of different levels, manages level-specific animations (like water), and provides methods to switch between levels.

Here is a breakdown of the key components and methods in the `LevelManager` class:

6. Main

A. Game:

The **Game** class represents the main game loop and handles the core logic for running and rendering the game. It controls the game's state (e.g., menu, playing, options), manages game resources, and initializes all the necessary components for the game to function.

Methods:

1. Game Loop:

- The game loop is started by the `startGameLoop()` method, which creates a new thread and runs the game loop in the `run()` method.
- The `run()` method is responsible for updating the game logic (`update()`) and rendering the game (`render()`) at a consistent FPS and UPS rate. It uses `System.nanoTime()` for time tracking to ensure smooth updates and rendering.

2. Game Update and Rendering:

- The `update()` method updates the game state based on the current game state (MENU, PLAYING, OPTIONS, CREDITS).
- The `render(Graphics g)` method draws the appropriate screen based on the current game state.

3. **Window Focus Lost:**

- The `windowFocusLost()` method is called when the window loses focus. It resets the player's direction booleans if the player is in the game state (PLAYING).

4. **Accessors for Game States:**

- Getter methods are provided to access different game states like menu, playing, credits, etc., allowing other parts of the game to interact with these states.

B. GamePanel:

The **GamePanel** class represents the main drawing surface for the game. It is a `JPanel` in which the game is rendered and where user input is processed. This panel is used to display all game content, including graphics and interactive elements.

C. GameWindow:

The **GameWindow** class is responsible for setting up and managing the game window in which the game is displayed. This class leverages `JFrame` from the `javax.swing` package to create a window with the appropriate configuration and behavior.

D. MainClass:

The **MainClass** is a simple entry point for starting the game. It contains the main method, which is the starting point of any Java application.

7. Object:

A. BackgroundTree and Cannon:

The **BackgroundTree** class in Java that represents a tree in a game or graphical environment. The class has the following features:

The **Cannon** class extends a **GameObject** class and seems to be part of a game where it represents a cannon object within a tile-based game world. Here's an analysis of the code:

C. GameContainer:

The **GameContainer** class you provided extends **GameObject** and appears to represent a container object (possibly a crate or similar object) in a game world. It is part of a tile-based or object-based game where containers might interact with other elements of the game, like being picked up or moved. Here's a breakdown of the code:

D. GameObject:

The **GameObject** class serves as a base class for various game objects, such as containers, cannons, and other interactive elements within the game world. This class handles key aspects such as position, animation, hitboxes, and interaction states. Here's a breakdown of the main components:

Use Case and Behavior:

- This class would be extended by specific game objects (e.g., **GameContainer**, **Cannon**, etc.), which inherit the core functionality provided by **GameObject**.
- **Animation**: The animation handling is designed to support different game objects that may or may not have animations. For example, a **Cannon** might have an animation for rotating, while a **Box** might have no animation.

- **Collision:** The hitbox is used for collision detection and interactions. It can be drawn for debugging or collision checking, and it determines whether other objects interact with it.
- **State Management:** The active flag and the doAnimation flag are essential for managing the object's interaction and animation status.

The **ObjectManager** class is responsible for managing and updating various game objects such as potions, containers, projectiles, cannons, spikes, and background elements like trees and grass. It interacts with other game components like Player, Enemy, and Level to manage gameplay events, object collisions, animations, and rendering.

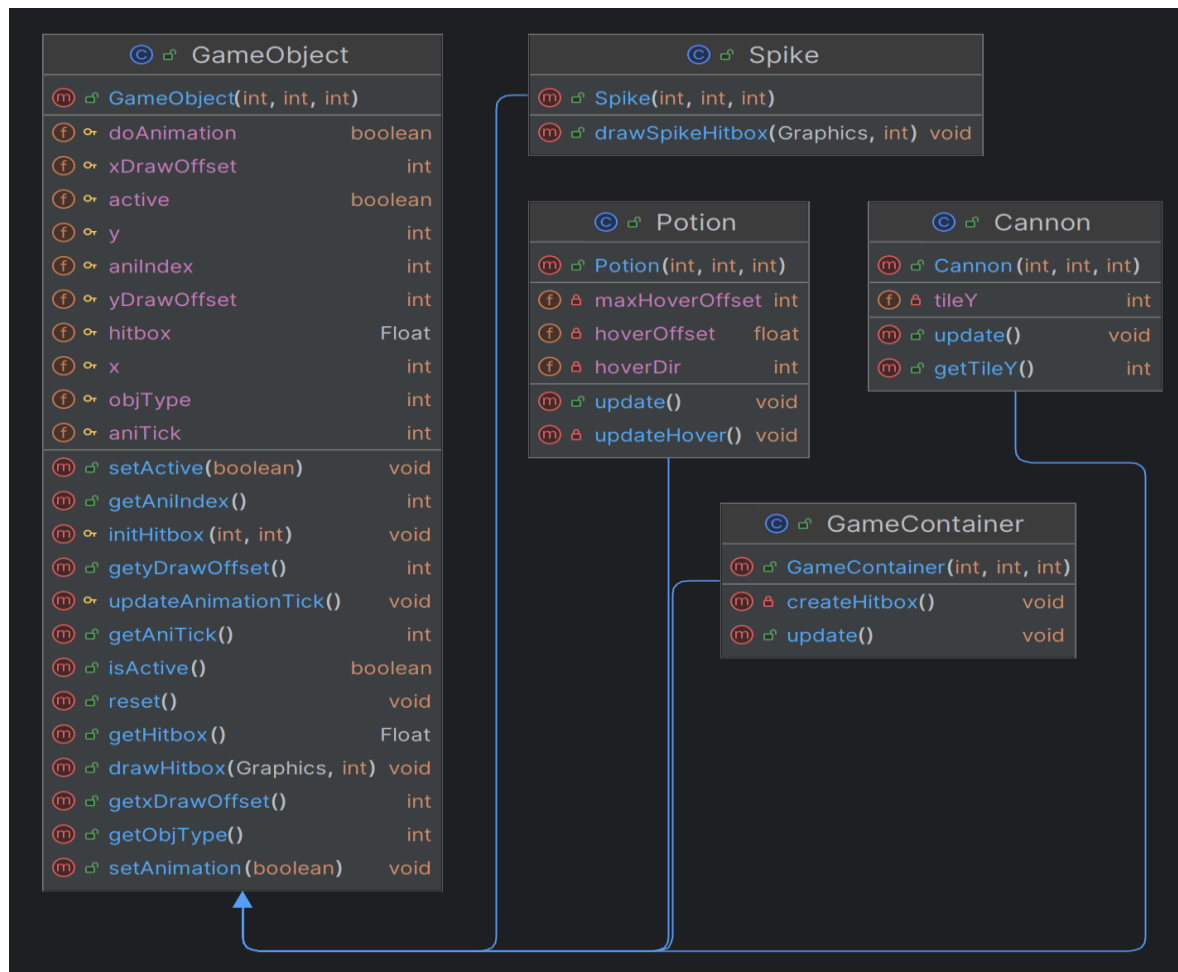


Figure 4: GameObject

3.8 UI:

1. PauseButton

- **Role:** Represents a button for pausing the game.
- Handles button-specific behavior like drawing, checking for mouse events, and initializing button properties.

2. SoundButton

- **Role:** Represents buttons related to controlling sound settings.
- Manages toggling sound on/off and checks for mouse events like hovering or pressing.

3. UrnButton

- **Role:** A general-purpose button with functionalities for mouse input handling and graphical rendering.
- Includes basic button behaviors like mousePressed, mouseReleased, and state updates.

4. AudioOptions

- **Role:** Controls audio settings within the game.
- Manages audio-related buttons (e.g., SoundButton) and handles user interaction with those options.

5. GameOverOverlay

- **Role:** Represents the overlay shown when the game ends.
- Manages buttons and input for the "game over" screen, such as mouse events.

6. VolumeButton

- **Role:** Represents a button to adjust volume levels.

- Handles user interactions like dragging to update volume, and mouse events for hovering or clicking.

7. **LevelCompletedOverlay**

- **Role:** Represents the overlay displayed when a game level is completed.
- Handles initialization, user input, and button interactions during the "level complete" state.

8. **PauseOverlay**

- **Role:** Represents the overlay shown when the game is paused.
- Manages pause screen buttons, mouse events, and graphical rendering.

9. **MenuButton**

- **Role:** Represents buttons used within the game menu.
- Handles button interactions like mouse events and renders the button states.

10. **GameCompletedOverlay**

- **Role:** Represents the overlay displayed when the entire game is completed.
- Manages buttons, mouse events, and initialization for the game completion screen.



Figure 5: UI

4. Conclusion

1. Future Work:

Our current application still has vital problems that the workers are hoping to address in future implementation. Here are some of them:

2. Implementing more User Interface (UI):

The user interface (UI) is a bit off since we had not got a user setting site, adding ticket, commenting page to function properly. We modified the login and sign up forms to store the user email in the database, had we not had abundant time to do everything we originally planned. Our team wanted to use ReactJS ⁽⁵⁾ Framework to make features like forms, buttons, and menus for easier user interaction.

3. Complete the security system:

The existing application is susceptible to security vulnerabilities due to the absence of decentralized security measures. This means that a customer's token, once acquired, grants unrestricted access and modification privileges to all data in the database, posing a significant risk if the token falls into the wrong hands.

In simple terms, unauthorized possession of a user's token could enable malicious manipulation of the database, resulting in potential property damage and complications. This flaw may lead to the exploitation of crucial information, precipitating serious legal and financial consequences. To address these security concerns, upcoming versions of the database will implement a stringent user access control policy, mitigating the risk of unauthorized access and manipulation.

4. Summary:

To conclude, despite some shortcomings, we have achieved the main goal of the project. We have followed all the use cases in our Use Case Diagram and created a Database that meets the Third Normal Form. If you want to learn how to use our application for your own project, please contact us!

Github:

https://github.com/alook123123/2DGame?fbclid=IwZXh0bgNhZW0CMTAAAR3c7Q00CquULrAwn5MbeD0zKcKxXOZVpB1XNclLqhkLSGwvQi0Sq_4Pu3Y_aem_1FbPjkiWzybGNv7ceTFbSq

Reference:

Kaarin Gaming Java Game Development Tutorial